

Research Plan: Formal Methods in Mathematics

Project Title: Computer-Assisted Verification: An Investigation into Proof Assistants and Formal Mathematical Reasoning

Student Name: William Fayers

Supervisor: Dr. Yuri Santos Rego

Email: YSantosRego@lincoln.ac.uk

Module: MTH3011/MTH3005

Submission Date: 27 October 2025

1. Project Overview and Motivation

Mathematics derives its foundational power from rigorous logical deduction. However, despite centuries of refinement, even published proofs by experienced mathematicians occasionally contain errors, gaps, or ambiguities that undermine their validity. The seminal example of the Needham-Schroeder authentication protocol—which remained unquestioned for nearly a decade before researchers discovered critical security flaws—demonstrates that logical errors can persist even in carefully scrutinized work. Similarly, monumental results such as the Feit-Thompson theorem span hundreds of pages of dense argumentation, making human verification extraordinarily challenging.

This project investigates **formal methods in mathematics**: the use of computer-assisted proof verification systems to establish mathematical truth with unprecedented rigor. Proof assistants such as Lean, Coq, and the Prototype Verification System (PVS) provide mechanized frameworks based on higher-order logic and dependent type theory, enabling mathematicians to construct machine-checkable proofs that eliminate ambiguity through computers' inherently boolean nature.

My research will explore the **logical foundations** of formal verification, examine **contemporary proof assistants** and their applications, and undertake **hands-on formalization** of mathematical theorems. This interdisciplinary work intersects mathematical logic, computer science, and formal verification—domains that align with my background as a mathematics undergraduate with extensive programming experience in Python and other languages.

Personal Motivation: Having developed complex software systems and witnessed firsthand the challenges of ensuring correctness in computational systems, I am particularly drawn to the intersection of mathematical rigor and computational verification. My experience with network infrastructure configuration and debugging has reinforced the importance of formal specifications and rigorous validation. This project offers an opportunity to explore how mathematical foundations can provide guarantees of correctness that testing alone cannot achieve.

2. Connection to Previous Studies

This project builds upon several strands of my academic and practical experience:

Mathematical Background:

- **MTH2001 - Analysis:** Foundations in epsilon-delta proofs and rigorous mathematical reasoning
- **MTH2002 - Linear Algebra:** Experience with abstract algebraic structures and formal mathematical argumentation
- **MTH2006 - Numerical Methods:** Computational approaches to mathematical problems, relevant to understanding automated reasoning
- **Current coursework:** Advanced topics in mathematical logic and computation

Programming and Computer Science:

- Extensive experience in **Python** with emphasis on clean code architecture and comprehensive documentation (Google-style formatting, NumPy docstrings)
- Proficiency in **TypeScript** and modern software development practices
- Experience with **Git**, **API integration**, and debugging complex systems
- Background in network infrastructure (Ubiquiti systems, configuration management) demonstrating systematic problem-solving

Relevant Skills for Formal Methods:

- Strong foundation in **mathematical proof techniques** from analysis and algebra modules

- **Functional programming concepts** through exposure to typed languages
- Experience with **formal specifications** in software development
- Systematic debugging methodology applicable to proof verification

The project directly leverages my programming expertise while deepening my understanding of mathematical foundations—a synergy particularly valuable given that proof assistants essentially require "programming" mathematical proofs in a rigorous formal language.

3. Research Objectives and Questions

This project aims to address the following research objectives:

Primary Objectives:

1. **Understand the logical foundations of formal mathematics**
 - Investigate propositional and predicate logic as bases for formal systems
 - Examine higher-order logic and dependent type theory
 - Study the Curry-Howard correspondence connecting logic and computation
2. **Explore incompleteness and its implications**
 - Analyze Gödel's incompleteness theorems and their philosophical significance
 - Understand completeness of formal systems and decidability boundaries
 - Investigate what formal methods can and cannot verify
3. **Master a contemporary proof assistant**
 - Gain proficiency in Lean 4 (primary) or Coq (secondary)
 - Learn the syntax, tactics, and proof strategies
 - Explore the mathlib library and its organizational principles
4. **Conduct hands-on formalization work**
 - Formalize selected mathematical theorems
 - Document the formalization process and challenges encountered
 - Compare informal mathematical reasoning with formal proof construction
5. **Investigate real-world applications**
 - Examine case studies of formal verification in industry (NASA, cryptographic protocols)
 - Analyze the verification of the Needham-Schroeder protocol flaws
 - Study large-scale formalization projects (Feit-Thompson theorem in Coq)

Research Questions:

- How do formal proof systems encode mathematical reasoning, and what guarantees do they provide?
- What are the practical challenges in formalizing "everyday" mathematical arguments?
- How does the expressiveness of different logical foundations (simple type theory vs. dependent types) affect formalization?
- What lessons from formal verification can inform better mathematical practice?
- How are automated reasoning and interactive theorem proving complementary?

4. Literature Survey

The following sources provide a comprehensive foundation for investigating formal methods in mathematics:

Core References (Provided in Project Brief):

1. **M. Ayala-Rincon & F. L. C. de Moura** (2017). *Applied Logic for Computer Scientists: Computational Deduction and Formal Proofs*. Springer.
Foundation in computational logic and automated deduction techniques.
2. **H. Geuvers** (2009). "Proof Assistants: History, Ideas and Future." *Sadhana*, 34, 3-25.
Historical overview and future directions of proof assistant development.
3. **L. C. Paulson** (2014). "A Machine-Assisted Proof of Gödel's Incompleteness Theorems for the Theory of Hereditarily Finite Sets." *Review of Symbolic Logic*, 7(3), 484-498.
Demonstration of formalizing deep metamathematical results.
4. **E. Mendelson** (2015). *Introduction to Mathematical Logic* (6th ed.). CRC Press.
Comprehensive treatment of formal logic fundamentals.

Extended Literature Base:

Proof Assistants and Theorem Proving:

5. **C. Paulin-Mohring** (LASER 2008). "Introduction to the Coq Proof-Assistant for Practical Software Verification." *Lecture Notes*.
Practical introduction to Coq's dependent type theory.
6. **A. Avigad** (2018). "Formal Methods in Mathematics and the Lean Theorem Prover." *Carnegie Mellon University Slides*.
Overview of Lean's approach to formalization.
7. **J. Avigad et al.** (2020). "The Lean Mathematical Library." *Proceedings of CPP 2020*.
Documentation of mathlib's comprehensive formalized mathematics.
8. **G. Gonthier et al.** (2013). "A Machine-Checked Proof of the Odd Order Theorem." *ITP 2013*.
Landmark formalization demonstrating scalability of proof assistants.

Foundations and Type Theory:

9. **G. Barthe** (LASER lectures). "Introduction to Dependent Type Theory and Higher-Order Logic."
Foundations of dependent types underlying modern proof assistants.
10. **P. Dybjer** (2016). "Intuitionistic Type Theory." *Stanford Encyclopedia of Philosophy*.
Philosophical and technical foundations of type-theoretic logic.
11. **HoTTbook** (2013). *Homotopy Type Theory: Univalent Foundations of Mathematics*.
Cutting-edge perspective on type theory as mathematical foundation.

Formal Verification Applications:

12. **E. Denney, B. Fischer, J. Schumann** (2004). "Using Automated Theorem Provers to Certify Auto-Generated Aerospace Software." NASA Technical Report.
Industrial application at NASA using PVS.
13. **L. C. Paulson** (1997). "Mechanizing Coinduction and Corecursion in Higher-Order Logic." Cambridge Computer Lab Technical Report.
Needham-Schroeder protocol verification revealing decade-old flaws.
14. **M. H. ter Beek et al.** (2023). "Formal Methods in Industry." *Software and Systems Modeling*.
Comprehensive survey of successful industrial applications.

Automated Reasoning and Incompleteness:

15. **J. Urban et al.** (2012). "MaLAREa SG1: Machine Learner for Automated Reasoning with Semantic Guidance." *IJCAR 2012*.
Machine learning approaches to automated theorem proving.
16. **K. Gödel** (1931). "Über formal unentscheidbare Sätze der Principia Mathematica und verwandter Systeme I." *Monatshefte für Mathematik und Physik*, 38, 173-198. (English translation available)
Original incompleteness theorem paper—historical significance.
17. **B. C. Pierce** (2002). *Types and Programming Languages*. MIT Press.
Comprehensive treatment of type systems and their logical interpretations.

Popular and Expository Sources:

18. **K. Buzzard** (2022-2025). *Xena Project Blog* and *Formalising Mathematics* course materials.
Accessible introduction to Lean for mathematicians.

19. **T. Hales** (2014). "Developments in Formal Proofs." *Séminaire Bourbaki.*
Survey of major formalization milestones including Kepler conjecture.

20. **Plus Magazine** (2025). "Proof Assistants (Part 2)." *University of Cambridge.*
Popular mathematics article on recent developments in formalization.

This literature base provides:

- **Foundational theory** (logic, type theory, incompleteness)
- **Practical tools** (Lean, Coq, PVS documentation and tutorials)
- **Case studies** (large formalizations, industrial applications)
- **Current research** (automated reasoning, AI-assisted proof)

5. Methodology and Action Plan

Research Approach:

This project employs a **mixed theoretical-practical methodology**:

Phase 1: Foundational Study (Weeks 1-8)

- Systematic study of mathematical logic and type theory from core texts
- Self-directed learning of proof assistant syntax and tactics
- Exploration of existing formalized libraries (mathlib)
- Documentation in research logbook of key concepts and insights

Phase 2: Practical Formalization (Weeks 9-20)

- Hands-on exercises: formalizing elementary theorems
- Progressive complexity: from propositional logic to analysis/algebra results
- Selection and formalization of a substantial case study
- Comparative analysis: informal vs. formal proof strategies

Phase 3: Advanced Investigation (Weeks 21-26)

- Deep dive into selected advanced topic (e.g., Gödel's theorems, protocol verification)
- Analysis of major formalization projects
- Reflection on the formal methods paradigm

Phase 4: Synthesis and Reporting (Weeks 27-30)

- Consolidation of findings
- Report writing and revision
- Preparation of formalized code repository
- Viva preparation

Detailed Timeline:

Week	Activity	Deliverables
1-4	Literature review, logic foundations study	Annotated bibliography, logbook entries
2-8	Learn Lean/Coq: syntax, tactics, basic proofs	Simple formalized proofs
5-8	Study formal logic, type theory, Curry-Howard	Conceptual summaries

Week	Activity	Deliverables
6	Logbook Submission 1	Completed logbook
9-12	Case study selection, initial formalization	Formalization progress
10-18	Intensive hands-on formalization work	Substantial formalized proofs
12	Logbook Submission 2	Updated logbook
14	Mid-Project Report	Progress report document
15-20	Advanced topics (incompleteness/verification)	Extended case studies
18	Logbook Submission 3	Final logbook updates
21-26	Draft report writing	Draft chapters
27-30	Report revision, code documentation	Final report
30	Final Report Submission (22 May 2026)	Complete scientific report
31+	Viva Preparation & Examination	Presentation materials

Supervisory Meetings:

Schedule: Fortnightly meetings (30 minutes) at mutually agreed times

Format:

- Progress updates and demonstration of formalized work
- Discussion of conceptual challenges encountered
- Guidance on literature and research direction
- Logbook review and supervisor signature

Preparation for meetings:

- Prepare specific questions in advance
- Bring formalized code examples for discussion
- Maintain up-to-date logbook with dated entries

6. Equipment, Facilities, and Resources

Software and Computational Resources:

Primary Tools:

- **Lean 4** with mathlib library (primary proof assistant)
- **Coq** with standard library (secondary/comparative study)
- **VS Code** with Lean/Coq extensions (development environment)
- **Git/GitHub** for version control of formalized code

Development Environment:

- Personal laptop/desktop computer (sufficient for proof assistant work)
- University computer laboratories (INB building) for additional access
- No specialized hardware required

Documentation and Writing:

- **LaTeX** (via Overleaf or local installation) for report preparation

- **Obsidian/Markdown** for research notes and logbook
- **Zotero or similar** for reference management

Facilities:

Primary workspace: University of Lincoln, Isaac Newton Building (mathematics department)

- Access to quiet study spaces
- Computer laboratory facilities
- Library resources (physical and electronic)

No special laboratory requirements as project is predominantly theoretical/computational

Cost Analysis:

Consumables: £0 (no physical materials required)

Software: £0 (all tools are free and open-source)

Equipment: £0 (using existing personal/university computers)

Total Project Cost: £0

This is a minimal-cost project relying entirely on freely available software and existing computational infrastructure.

7. Risk Assessment

Given the theoretical and computational nature of this project, health and safety risks are minimal and comparable to everyday office/study activities. However, the following considerations apply:

Identified Risks and Control Measures:

Task	Hazard	Who Affected	Probability	Severity	Initial Risk	Control Measures	Residual Risk
Extended computer work	Eye strain, repetitive strain injury	Student	3 (Probable)	2 (Minor)	6 (Medium)	Take 20-minute breaks every hour; follow DSE guidelines; maintain proper posture; adjust screen brightness	$1 \times 2 = 2$ (Low)
Computer use	Electrical hazard	Student and others nearby	1 (Extremely remote)	4 (Major)	4 (Low)	PAT-tested equipment only; no liquids near equipment; proper cable management	$1 \times 4 = 4$ (Low)
Prolonged sitting	Musculoskeletal discomfort	Student	2 (Possible)	2 (Minor)	4 (Low)	Regular breaks; stretching; ergonomic seating; standing desk option	$1 \times 2 = 2$ (Low)

Task	Hazard	Who Affected	Probability	Severity	Initial Risk	Control Measures	Residual Risk
Mental workload	Stress, cognitive fatigue	Student	2 (Possible)	2 (Minor)	4 (Low)	Structured work schedule; regular supervisor meetings; maintain work-life balance	$1 \times 2 = 2$ (Low)

Overall Risk Assessment: Low (all residual risks ≤ 4)

Additional Safety Considerations:

- Comply with Display Screen Equipment (DSE) guidance from university Health & Safety induction
- Ensure adequate ventilation in workspace
- Maintain good lighting to reduce eye strain
- Keep workspace organized to prevent trip hazards from cables

Emergency Procedures:

- Familiar with building emergency exits and assembly points
- First aid facilities location in Isaac Newton Building
- University security contact: available via campus phone system

No specialized risk assessment required beyond standard office/computer work precautions, as confirmed with supervisor. This assessment will be reviewed if project scope changes to include any physical experiments or equipment testing (not currently anticipated).

8. Ethical Considerations

This project involves theoretical research and computational work using publicly available software and published mathematical literature.

Ethics Approval Status: Not required

Rationale:

- No human subjects or data collection from individuals
- No animal subjects
- No access to sensitive/confidential information
- All source materials are published academic work in the public domain
- Software used is open-source and publicly available

Should the scope expand to include user studies (e.g., evaluating mathematician interactions with proof assistants), appropriate ethics approval will be sought via the College ethics committee before proceeding.

9. Expected Outcomes and Contributions

Anticipated Deliverables:

1. Comprehensive Literature Review

Synthesis of formal methods theory, proof assistant capabilities, and applications

2. Repository of Formalized Mathematics

Collection of machine-verified proofs demonstrating practical skills

- Elementary results in logic, algebra, or analysis
- At least one substantial theorem formalization

- Well-documented code with explanatory comments

3. Case Study Analysis

Detailed examination of a significant formalization project or application

- Needham-Schroeder protocol verification, OR
- Portion of a major theorem formalization, OR
- Automated reasoning application

4. Critical Evaluation

Reflective analysis of:

- Strengths and limitations of formal methods
- Comparison of informal vs. formal mathematical practice
- Future directions for proof assistants
- Implications for mathematical rigor and education

5. Scientific Report

~10,000-word report following academic standards with:

- Clear exposition of foundational concepts
- Documentation of hands-on work
- Critical analysis grounded in literature
- Proper citations and bibliography

Potential Project Extensions:

If time permits and progress is favorable:

- Formalization of a more advanced result
- Comparative study of multiple proof assistants (Lean vs. Coq)
- Investigation of AI-assisted theorem proving (emerging area)
- Contribution to mathlib or similar open-source project

Broader Impact:

This project contributes to:

- **Personal development:** Deep understanding of mathematical foundations and computational reasoning
- **Academic skills:** Rigorous logical thinking, formal specification, technical writing
- **Career preparation:** Skills applicable to formal verification in industry, research in logic/CS, or teaching rigorous mathematics
- **Mathematical community:** Documentation of the formalization experience valuable to others exploring proof assistants

10. Conclusion

This research plan outlines a rigorous investigation into formal methods in mathematics, balancing theoretical foundations with practical hands-on experience. By engaging with proof assistants, I will develop both technical skills in formalization and deeper insight into the logical structure of mathematics itself.

The project directly addresses key learning objectives:

- **Independent research:** Self-directed learning of proof assistants and exploration of formal verification
- **Critical analysis:** Evaluation of formal methods' strengths, limitations, and applications
- **Technical proficiency:** Mastery of specialized tools (Lean/Coq) and formalization techniques
- **Professional communication:** Documentation via logbook and scientific report

I am enthusiastic about this project's potential to bridge my interests in mathematics, computer science, and rigorous reasoning. The skills developed—formal specification, logical analysis, and systematic verification—are increasingly relevant in both academia and industry, particularly in safety-critical systems and trustworthy AI.

I look forward to regular discussions with Dr. Santos Rego throughout this research journey and to contributing to the growing community of formal mathematics.

Signatures:

Student: _____ Date: 27 October 2025

Supervisor: _____ Date: _____

References

[References formatted in IEEE style as required by handbook]

- [1] M. Ayala-Rincon and F. L. C. de Moura, *Applied Logic for Computer Scientists: Computational Deduction and Formal Proofs*. Springer, 2017.
- [2] H. Geuvers, "Proof assistants: history, ideas and future," *Sadhana*, vol. 34, pp. 3-25, 2009.
- [3] L. C. Paulson, "A machine-assisted proof of Gödel's incompleteness theorems for the theory of hereditarily finite sets," *Review of Symbolic Logic*, vol. 7, no. 3, pp. 484-498, 2014.
- [4] E. Mendelson, *Introduction to Mathematical Logic*, 6th ed. CRC Press, 2015.
- [5] C. Paulin-Mohring, "Introduction to the Coq proof-assistant for practical software verification," LASER 2008 lecture notes.
- [6] A. Avigad, "Formal methods in mathematics and the Lean theorem prover," Carnegie Mellon University, 2018.
- [7] The mathlib Community, "The Lean mathematical library," in *Proc. 9th ACM SIGPLAN Int. Conf. Certified Programs and Proofs (CPP 2020)*, pp. 367-381, 2020.
- [8] G. Gonthier et al., "A machine-checked proof of the odd order theorem," in *Interactive Theorem Proving (ITP 2013)*, Springer LNCS 7998, pp. 163-179, 2013.
- [9] G. Barthe, "Introduction to dependent type theory and higher-order logic," INRIA Sophia-Antipolis lecture notes.
- [10] P. Dybjer, "Intuitionistic type theory," *Stanford Encyclopedia of Philosophy*, E. N. Zalta, Ed., 2016.
- [11] Homotopy Type Theory: Univalent Foundations of Mathematics. Institute for Advanced Study, 2013.
- [12] E. Denney, B. Fischer, and J. Schumann, "Using automated theorem provers to certify auto-generated aerospace software," NASA Ames Research Center Technical Report, 2004.
- [13] L. C. Paulson, "The inductive approach to verifying cryptographic protocols," *Cambridge Computer Lab Technical Report UCAM-CL-TR-413*, 1997.
- [14] M. H. ter Beek, M. Huisman, P. Monti, J. Peleska, and J. van de Pol, "Formal methods in industry," *Software and Systems Modeling*, vol. 22, pp. 1319-1329, 2023.
- [15] J. Urban, J. Vyskočil, and P. Štěpánek, "MaLAREa SG1: Machine learner for automated reasoning with semantic guidance," in *Automated Reasoning (IJCAR 2012)*, Springer LNCS 7364, pp. 441-456, 2012.
- [16] K. Gödel, "Über formal unentscheidbare Sätze der Principia Mathematica und verwandter Systeme I," *Monatshefte für Mathematik und Physik*, vol. 38, pp. 173-198, 1931.
- [17] B. C. Pierce, *Types and Programming Languages*. MIT Press, 2002.
- [18] K. Buzzard, "Xena project blog," [Online]. Available: <https://xenaproject.wordpress.com/>
- [19] T. Hales, "Developments in formal proofs," *Séminaire Bourbaki*, vol. 1086, 2014.
- [20] "Proof assistants (part 2)," *Plus Magazine*, University of Cambridge, 2025. [Online]. Available: <https://plus.maths.org/>

Appendix A: Project Timeline (Gantt Chart)

See attached CSV file: `project_timeline.csv`

Appendix B: Proof Assistant Installation Guide

[To be completed during project: documented steps for setting up Lean 4 development environment]

Appendix C: Formalized Code Repository

[To be completed during project: GitHub repository link and documentation]

End of Research Plan